

HELIOS_PROJECT_BIBLE_PLAIN

Helios Project Bible — In Plain English

Plain-language companion to `docs/architecture/HELIOS_PROJECT_BIBLE.md` . For the exact spec, read that / its PDF.

In one sentence

Helios is an always-on robot growth analyst that watches an online store's sales funnel, figures out *why* a number moved (not just *that* it moved), puts a dollar figure on it, suggests an experiment to fix it, and writes the boss a short report — all by itself, on a schedule.

Why this matters to you

If you understand this doc, you can build the whole thing. The genius of Helios isn't fancy AI — it's the *guardrails*. The language model (LLM) is kept on a very short leash: it picks names from a menu and calls trusted tools, but it never writes database queries or does math itself. Once you grasp that one idea, every other piece (the data tables, the tool servers, the agents, the test suite) is just a way of enforcing it. Build that leash correctly and you have a system people actually trust.

The big ideas, simply

The funnel. A shopper moves through stages: `session_start` → `view_item` → `add_to_cart` → `begin_checkout` → `add_shipping_info` → `add_payment_info` → `purchase` . Helios tracks the drop-off at each step. The headline number is `session_conversion_rate` (the share of visits that end in a purchase).

Mix-shift vs rate-change — the heart of it. Say conversion drops. Two very different reasons:

- **rate-change** (rate effect): people in a group actually started buying less. A *real* behavior problem.

- **mix-shift** (mix effect): each group buys at the same rate as before, but more of your traffic shifted toward a group that always buys less. Composition changed, behavior didn't.

A naive analyst sees the drop and "fixes checkout" — wrong, if it was just mix-shift (a trap called **Simpson's paradox**, where the total moves opposite to every group). Helios splits every change into **mix effect + rate effect + interaction** (the third being when both move together) with one exact formula, so it chases real behavior changes, not composition artifacts.

Grounding over generation — the two iron rules.

1. The LLM *never writes SQL by hand*. To get data it asks a tool for a pre-approved metric by name. Invent a fake column and it errors out. Result: zero hallucinated columns.
2. The LLM *never does math in words*. Every statistic runs in real, seeded Python code. Numbers in the report are copied verbatim from those tools.

Dollars, always. A "2% drop" isn't a decision until it's "\$14k of revenue-at-risk." Every finding carries a dollar figure (roughly: rate change × affected sessions × value).

Proactive, not reactive. Helios's heartbeat is the scheduled automatic run that ships a **Decision Brief** before anyone asks. Chatting with it is a minor side feature, not the point.

What you actually build

Roughly in order:

1. **The data spine (dbt).** A four-layer pipeline turning raw Google Analytics 4 (GA4) events into clean tables: staging → intermediate → marts. The keystones are sessionization (stitching events into sessions via the `(user_pseudo_id, ga_session_id)` key) and the `reached_*` funnel flags. Get these wrong and every downstream number is silently wrong.
2. **The semantic layer.** A YAML registry where every metric (like `session_conversion_rate`) is defined once, owned, and versioned. This is the menu the LLM orders from.
3. **The five MCP tool servers** (MCP = Model Context Protocol, a standard way for an AI to call typed tools):
 - `warehouse-mcp` — the *only* thing that can run a query; checks cost first.
 - `semantic-mcp` — the *only* path to SQL; turns metric names into safe queries.
 - `stats-mcp` — the *only* path to math; does the mix/rate/interaction split, significance tests, forecasts.
 - `experiment-mcp` — sizes and designs the suggested experiments.
 - `report-mcp` — writes the brief and remembers past runs.
4. **The seven agents** (Claude Agent SDK): **Orchestrator** (plans the run), **Monitor** (spots anomalies), **Decompose** (splits mix vs rate), **Diagnose** (hunts the root cause), **Prescribe**

(designs experiments), **Narrator** (writes the brief), and **Critic** (an adversary that tries to *disprove* every finding before it ships). Each agent only gets the tools it needs.

5. **The eval benchmark.** Fifty test cases where you secretly inject a known problem, then check whether Helios rediscovers it. Target: 85% correct vs ~45% for a naive analyst, with zero hallucinations. This runs in CI and is the regression firewall.

Easy things to get wrong

- **Silent data bugs.** Bad sessionization or funnel flags don't crash — they just quietly produce wrong numbers. Lock them down with golden-value tests.
- **Averaging rates.** Never average per-group rates; re-sum the numerator and denominator. Averaging *causes* Simpson's paradox.
- **The `traffic_source` trap.** GA4's `traffic_source` is the user's *first-ever* touch, not this session's source. Prefer session-scoped source/medium.
- **Scope creep into the anti-product.** Helios is *not* a dashboard, *not* an ask-your-data chatbot, *not* an ad-hoc SQL tool. If you build those, you've built the wrong thing.
- **Letting the LLM off the leash.** Any hand-written SQL or in-prose statistic breaks the whole trust model.

Glossary — the exact words, demystified

- `session_conversion_rate` — share of sessions that purchased ($\text{purchasing_sessions} / \text{sessions}$).
- **mix effect / rate effect / interaction** — the three parts a metric change splits into: composition shifted / behavior shifted / both shifted together.
- `warehouse-mcp` — sole query runner; enforces the cost budget.
- `semantic-mcp` — sole SQL author; the anti-hallucination chokepoint.
- `stats-mcp` — sole math engine; deterministic, seeded.
- `experiment-mcp` — designs and sizes experiments.
- `report-mcp` — renders the Decision Brief and stores memory.
- **Orchestrator / Monitor / Decompose / Diagnose / Prescribe / Narrator / Critic** — the seven agents; the Critic is the adversary that gates every finding.
- **Decision Brief** — the executive report; the actual product.
- **revenue-at-risk** — the dollar size of a finding.
- **dry_run** — a cost/schema check that runs before any real query.
- **reconcile** — checks a number against canonical totals (must be within 0.5%).
- **governed SQL** — queries written only by the semantic layer, never by the LLM.

When to open the real doc

This companion skips the depth: the full 25-section spec has every metric's exact SQL, the dbt project tree and tests, the channel-grouping rules, the YAML field contracts, per-tool input/output schemas, the agent state machine and pruning thresholds, the memory architecture, requirement IDs (FR-/NFR-), and the experiment/roadmap framing. When you need the precise, buildable detail — open `docs/architecture/HELIOS_PROJECT_BIBLE.md` or the PDF at `pdf/docs/architecture/HELIOS_PROJECT_BIBLE.pdf` .